PRODUCT REQUIREMENTS DOCUMENT

AI-Powered Financial Schema Generation System

Automating JSON Schema Authoring via RAG, 4C Validation & LLM Reasoning

Document Version	1.0 — Initial Draft
Status	For Stakeholder Review
Prepared By	AI Platform Engineering
Date	May 2026
Classification	Internal — Confidential
Primary Audience	Product, Engineering, Domain SMEs, Finance Ops
Schema System	4C Engine — SC00004 (PO Validation) used as reference

1. Executive Summary

This document defines requirements for an AI-powered system that automates the creation of JSON schemas for a FinTech computation engine. The engine accepts schemas as input, processes them against source financial data, and produces computed outputs — covering problems such as purchase order valuation, cost allocation, workflow generation, payroll, budgeting, and more.

Today, every schema is authored manually by engineers who must understand the 4C structural contract (Class, Components, Criteria, Conditions), know which data sources and computation patterns apply, and correctly wire cross-schema dependencies. A schema for a complex problem may contain 19 or more schema classes, each with multiple components and criteria — representing days of expert engineering effort per problem.

The proposed system treats the existing library of 30 validated schemas as a knowledge corpus. When a new financial computation problem arrives in natural language, the system retrieves the most structurally and semantically similar schema classes from the corpus, adapts them to the new problem's parameters, assembles them in the correct dependency order, validates the output against 4C structural rules, runs a dry-run through the engine, and presents a draft schema for human review and approval.

The result is a supervised automation loop that eliminates repetitive manual schema authoring, reduces error rates, and accumulates domain knowledge over time.

2. Background — The 4C Schema Contract

Every schema fed to the computation engine is a structured JSON document. Understanding its exact anatomy is mandatory before the automation system can be designed. This section documents the 4C structure as observed in the real SC00004 (PO Validation) schema.

2.1 Schema Root Structure

A schema is a JSON array with a single `Root` node. The root contains two children:

- **SchemaGroupName:** A string naming the schema family (e.g., `"PO Validation"`).
- **SchemaClass:** An array of schema class nodes, numbered `#SchemaClass#1`, `#SchemaClass#2`, etc. Each node represents one computational sub-problem.

The SC00004 schema contains 7 schema classes: PO_ItemCalculation, CostAllocationMaster, POCostAllocation1, TCostAllocationValue, POCostAllocation, AP, and POWF. These execute in dependency order, with later classes consuming outputs of earlier ones.

2.2 The 4 C's — Anatomy of a Schema Class

Each schema class node must contain exactly the following layers, which together form the '4C contract' this system is built around:

C	Name	What it is / What the engine uses it for
C1	Class	The schema class node itself. Carries class-level metadata: SchemaName (unique identifier), DataSource (which raw data table to read — e.g. PO_I, CA, AP), DateAggregationOn (the date field used for period grouping), SettlementType (how results are aggregated — e.g. 'Per LOB Monthly', 'Per lineid Monthly'), and OutputProcessCode (pipeline routing code, e.g. 'TT').
C2	Conditions	Field-level filter declarations on the class node itself (siblings of Components). Each condition specifies a field name, a value or wildcard ('*' means pass-through from engine input), and an equality operator (EQ, IN, LE, GE, NE). Conditions can also use the GETGROUPFROMSCHEMA2() expression to dynamically expand a dimension (LOB, Function, ActivityCode, etc.) by querying another schema class — this is the primary mechanism for cross-schema data joins at the class filter level.
C3	Components	The computation payload. A class contains one or more Component nodes (#Components#0, #Components#1, ...). Each component defines: ComponentName (output column name), ModelID (source schema for cross-refs, or 'MATH'), Type (MATH, FETCHFROMSCHEMA,

		SUMFROMSCHEMA), ComponentType (e.g. 'COST'), and ComponentExpr (the arithmetic expression, field reference, or cross-schema fetch JSON for this component).
C4	Criteria	Per-component filter, nested inside each Component node under the Criteria key. A Criteria block contains one or more #Criteria#N rows, each specifying the field conditions that must match for this component's value to be included in the computation. Criteria allow the same class to produce different component values for different data slices.

2.3 Component Types

The Type field inside each Component node determines how the engine resolves the component value:

- **MATH:** ComponentExpr is an arithmetic expression evaluated against fields of the class's own DataSource. Example: `(Qty * Rate) - DiscountAmt + AddlCharge`.
- **FETCHFROMSCHEMA:** ComponentExpr is a JSON join specification. The engine fetches a single value from a named upstream schema class by matching the specified join keys. ModelID identifies the source schema class.
- **SUMFROMSCHEMA:** Like FETCHFROMSCHEMA, but aggregates (sums) all matching values from the upstream schema class instead of fetching a single row.

2.4 Condition Expressions — GETGROUPFROMSCHEMA2

`GETGROUPFROMSCHEMA2(SchemaName;ColumnName;{filter_json})` is a special expression used in Condition fields (C2) to dynamically expand a dimension from another schema class. For example, the POWF class uses this to pull the list of applicable LOB values from CostAllocationMaster, and the list of ActivityCode values from AP — so the workflow class is automatically scoped to the correct LOBs and approval steps without hardcoding them.

2.5 Cross-Schema Dependency Pattern (SC00004 Example)

The dependency flow in SC00004 illustrates how classes chain together:

- **PO_ItemCalculation** (independent) — computes raw PO line item monetary values from PO_I data source.
- **CostAllocationMaster** (independent) — reads cost allocation percentages from CA data source.
- **POCostAllocation1** — fetches CostAllocationParameterValue from CostAllocationMaster via FETCHFROMSCHEMA; uses GETGROUPFROMSCHEMA2 on CostAllocationMaster to expand LOB and AllocationType dimensions.
- **TCostAllocationValue** — sums CostAllocationParameterValue from POCostAllocation1 via SUMFROMSCHEMA, aggregated by CostAllocationMethod.
- **POCostAllocation** — combines TCostAllocationValue (fetch), POCostAllocation1 sum, and PO_ItemCalculation sum to compute final allocated amounts per LOB.
- **AP** — reads approval workflow strategy data from AP data source.

- **POWF** — assembles the full workflow definition, pulling LOBs from CostAllocationMaster and activity details from AP via GETGROUPFROMSCHEMA2.

3. Problem Statement

3.1 Current State

- All 30 schemas are authored entirely by hand. Engineers must know the JSON structure, 4C rules, computation patterns, and cross-schema wiring from memory or informal documentation.
- A complex schema with 19 classes requires multiple days of expert engineering effort, followed by manual testing against the engine.
- There is no structured, searchable library of existing schemas. When a similar problem appears, engineers re-discover patterns from scratch or rely on informal knowledge transfer.
- Errors in schema authoring — missing Criteria filters, incorrect ComponentExpr syntax, wrong cross-schema join keys, or invalid SettlementType values — are not caught until engine execution, which can be a full batch cycle later.
- Schema knowledge is concentrated in a small number of senior engineers, creating a bottleneck and a key-person risk.

3.2 Root Causes

- No machine-readable representation of the 30-schema corpus that can be queried for structural similarity.
- No formal taxonomy of computation patterns (MATH chains, cost allocation splits, workflow assembly, etc.) that would allow pattern-level retrieval.
- No automated enforcement of 4C structural rules — the only validator is the engine itself, which runs downstream.
- No dependency graph that makes cross-schema relationships traversable by a tool.

3.3 Opportunity

The existing 30 schemas contain a finite set of recurring structural patterns. Analysis of SC00004 alone reveals four distinct patterns: sequential arithmetic chains (PO_ItemCalculation), master data fetch/normalize (CostAllocationMaster, AP), multi-step aggregation and allocation (POCostAllocation1/TCostAllocationValue/POCostAllocation), and workflow assembly (POWF). These patterns appear — with parametric variations — across the full corpus. An AI system that can recognize, retrieve, adapt, and correctly assemble these patterns can automate the large majority of new schema creation work.

4. Objectives & Success Criteria

4.1 Primary Objectives

- Build a queryable knowledge base from all 30 existing schemas, structured around the 4C anatomy.
- Enable natural language problem intake that produces a validated, structured problem specification before any generation begins.
- Automate schema class retrieval, 4C-compliant adaptation, dependency resolution, and JSON assembly for new problems.
- Guarantee structural correctness (4C rule compliance) before any draft schema reaches human review.
- Establish a feedback loop where approved schemas and reviewer corrections continuously improve the corpus.

4.2 Success Metrics

Metric	Target	Measurement
Schema authoring time — complex (10+ classes)	**< 4 hours**	Wall-clock from problem submission to human-approved schema
Schema authoring time — simple (< 5 classes)	**< 45 minutes**	Wall-clock for HIGH confidence, all-primitive schemas
4C structural validation pass rate	**> 99%**	Zero 4C violations in schemas reaching human review gate
Retrieval precision@5 (correct class in top 5)	**> 0.85**	Evaluated on 30-schema annotated holdout test set
Engine dry-run pass rate	**> 90%**	Assembled schemas producing correct output on synthetic test cases
Human escalation rate (LOW confidence classes)	**< 25%**	Fraction of generated classes requiring human edit
Corpus growth	**+10 schemas / quarter**	Approved schemas automatically added to knowledge base

5. System Overview

5.1 High-Level Flow

The system operates as a six-stage pipeline. Each stage has a well-defined input, output, and failure mode. The pipeline is deterministic and auditable — every LLM call, retrieval result, and validation outcome is logged.

Stage	Name	Input → Output	Failure Mode
0	Corpus Ingestion	30 raw schema JSON files → PostgreSQL + Neo4j + Pinecone	Blocked on SME annotation; no downstream impact
1	Problem Intake	Natural language problem → Structured Grounding Document	Ambiguities detected → halt; route to human clarification
2	Decomposition	Grounding Document → Ordered slot list (one slot per class needed)	Slot count / types incorrect → human review of decomposition
3	Retrieval & Selection	Slot list → Candidates per slot with confidence tiers	LOW confidence / GAP → flag for net-new generation
4	Adaptation & Assembly	Selected classes + Grounding Doc → Adapted 4C-valid class JSON → Assembled schema	4C validation failure → LLM retry (max 3) → escalate
5	Validation & Review	Draft schema → Engine dry-run result + Confidence report → Human review	Dry-run fail or reviewer rejects → restart from Stage 3

5.2 Infrastructure Components

Component	Technology	Role in the System
Vector Store	**Pinecone**	Semantic similarity search. Each schema class is embedded (as NL summary + metadata) and stored here. Used in Stage 3 retrieval.
Graph Database	**Neo4j**	Stores the dependency graph of all schema classes (FEEDS_INTO edges). Used for topological sort during assembly and cross-schema reference resolution.
Relational Store	**PostgreSQL**	Source of truth for raw schema JSON, parsed class records, reviewer edits, audit logs, and confidence reports.

Embedding Model	**voyage-finance-2**	Domain-tuned financial text embeddings. Substantially outperforms general models on financial schema retrieval tasks.
LLM Reasoning	**Claude Sonnet 4**	Structured extraction, decomposition, candidate selection, class adaptation, gap generation, and confidence report synthesis.
4C Validator	**Pydantic v2**	Automated structural rule enforcement: checks all 4C fields, component types, criteria nesting, and expression syntax before assembly.
Review Dashboard	**Streamlit v1**	Human review UI: confidence table, JSON diff view, engine result display, approve/edit/reject actions.
Pipeline Orchestration	**Prefect**	Stage sequencing, retry logic, failure escalation, and full audit logging of every pipeline run.

6. Detailed Functional Requirements

Stage 0 — Corpus Ingestion (Offline, One-Time)

This stage must be completed before any other stage is built. It converts the 30 raw schema JSON files into the three memory stores that power all downstream reasoning.

FR-0.1 Manual Schema Annotation

A domain SME must produce a structured metadata card for each schema and each class within it before ingestion begins. This annotation is the ground truth that determines retrieval accuracy.

Required annotation per schema:

- **Schema-level:** Schema ID, Schema Name, Problem Domain (e.g. Procurement/Payroll), Data Sources used, Number of classes, plain-English problem description, which classes are reusable primitives across schemas.
- **Class-level:** Class name, Computation Type (from controlled enum in §6 Appendix), primary DataSource, input field list, output field list, upstream dependencies, downstream consumers, cross-schema reference mechanisms used.

Annotation must be saved as structured YAML or JSON files, one per schema. These files are the input to all three ingestion sub-steps below. Estimated effort: 2–3 engineer-days with a domain SME.

FR-0.2 Schema Parsing — PostgreSQL Population

A Python parser traverses each raw schema JSON and extracts:

- All class-level metadata: SchemaName, DataSource, DateAggregationOn, SettlementType, OutputProcessCode.
- **Condition fields (C2):** Every non-standard child of the class node is a condition field. Parser must detect and record: field name, value (literal, wildcard `*`, or value list), equality operator (EQ, IN, LE, GE, NE).
- **GETGROUPFROMSCHEMA2 expressions:** When a condition field value matches the pattern `GETGROUPFROMSCHEMA2(SchemaName;Column;{filter_json})`, the referenced schema name is extracted and recorded as a dependency.
- **Components (C3):** For each component: ComponentName, ModelID, Type (MATH/FETCHFROMSCHEMA/SUMFROMSCHEMA), ComponentType, ComponentExpr. If Type is FETCHFROMSCHEMA or SUMFROMSCHEMA, the ModelID is recorded as a dependency.
- **Criteria (C4):** For each component, all #Criteria#N rows are extracted with their field names, values, and equality operators.

All extracted records are stored in PostgreSQL. The raw class JSON is stored alongside structured fields for use in adaptation (Stage 4).

FR-0.3 Dependency Graph — Neo4j Population

- Build a directed graph where nodes are schema class names and edges are FEEDS_INTO relationships.
- Two edge sources: (a) FETCHFROMSCHEMA / SUMFROMSCHEMA component references — ModelID → current class; (b) GETGROUPFROMSCHEMA2 condition references — referenced schema → current class.
- Each node stores: data_source, settlement_type, source_schema, computation_type.
- Each edge stores: ref_type (FETCHFROMSCHEMA / SUMFROMSCHEMA / GETGROUPFROMSCHEMA2), source component or condition field name.
- The graph must be cycle-free (DAG). A cycle detection check runs after every ingestion batch. Any detected cycle raises an ingestion error for manual resolution.

FR-0.4 NL Summary Generation & Vector Store Population

- For each class record, call the LLM to generate a 2–3 sentence technical summary describing: what the class computes, its DataSource, its key inputs/outputs, what Component Types it uses, and what downstream classes consume it.
- Embed each class using a combined text: NL summary + DataSource + SettlementType + computation_type + input field names + output field names.
- Use `voyage-finance-2` for embedding. Store vectors in Pinecone with metadata: schema_name, source_schema, data_source, settlement_type, computation_type, summary, depends_on, cross_ref_types.
- Vector ID format: `{source_schema}_{schema_name}` — ensures deterministic upserts when schemas are updated.

Stage 1 — Problem Intake & Grounding Document

FR-1.1 Natural Language Problem Intake

The system accepts a free-text problem statement from an operator or domain user describing a new financial computation scenario. No structured input is required from the user — the system extracts all structure.

FR-1.2 Structured Extraction — Grounding Document

The LLM extracts a structured Grounding Document from the problem statement. This document is the authoritative problem specification for all downstream stages. Required fields:

- **problem_domain:** PurchaseOrder | Invoice | Payroll | Budget | CostAllocation | Other
- **data_sources_mentioned:** List of recognized data source codes (PO_I, CA, AP, etc.)
- **computation_types_required:** List from controlled enum (sequential_math_chain, fetch_lookup, sum_aggregation, group_expansion, workflow_assembly, etc.)
- **lob_constraints:** Line of Business filters explicitly stated in the problem
- **function_constraints:** Approval function or workflow routing constraints

- **settlement_granularity:** The expected settlement level (per lineid, per LOB, per strategy, etc.)
- **special_conditions:** Non-standard filters or computation rules stated in the problem
- **ambiguities:** List of questions the system cannot resolve from the problem statement alone
- **estimated_complexity:** low (1–4 classes) | medium (5–10 classes) | high (11+ classes)

Critical rule: If the ambiguities list is non-empty, the pipeline HALTS immediately. The ambiguities are surfaced to the user for clarification. The pipeline does not proceed with any unresolved ambiguity. This is a hard requirement for financial schema correctness.

Stage 2 — Problem Decomposition

FR-2.1 Slot-Based Decomposition

Using the Grounding Document and the corpus taxonomy, the LLM decomposes the problem into an ordered list of schema class slots. Each slot corresponds to one schema class that the final schema will need.

Required fields per slot:

- **slot_id:** Sequential identifier (SLOT_01, SLOT_02, ...)
- **purpose:** Plain English description of what this class must compute
- **computation_type:** From the controlled enum
- **primary_data_source:** Expected DataSource for this class
- **expected_settlement_type:** Based on Grounding Document granularity
- **depends_on_slots:** List of slot_ids this slot consumes
- **is_likely_primitive:** True if this slot likely maps exactly to an existing corpus class unchanged
- **cross_schema_ref_types:** List of reference mechanisms expected (FETCHFROMSCHEMA, SUMFROMSCHEMA, GETGROUPFROMSCHEMA2)

Rules: slots must be ordered with independent slots first. Slots must not be invented without grounding in the problem statement. The decomposition is shown to the user for confirmation before retrieval begins.

Stage 3 — Retrieval & Candidate Selection

FR-3.1 Per-Slot Vector Retrieval

- For each slot, construct a query text from: purpose, computation_type, primary_data_source, expected_settlement_type, cross_schema_ref_types.

- Query Pinecone for top-K=5 candidates. Apply metadata filter on `computation_type` when non-null for higher precision.
- Return candidates with: similarity_score, schema_name, source_schema, summary, depends_on, metadata.

FR-3.2 Confidence Tiering

Each candidate is assigned a confidence tier based on similarity score (default thresholds, configurable):

- **HIGH (≥ 0.92):** The candidate is a near-exact match. If is_likely_primitive is also true, use the class as-is with no adaptation.
- **MEDIUM (0.82–0.91):** Good structural match but parametric differences (different LOB, different field names, different tax logic). Requires LLM-guided adaptation.
- **LOW / GAP (< 0.82):** No suitable match in corpus. Slot is flagged for net-new generation using corpus primitives as structural templates.

FR-3.3 LLM-Assisted Selection (MEDIUM Tier)

For MEDIUM confidence slots, the LLM reasons over the top-3 candidates and selects the best match. The selection prompt includes:

- The full slot requirement and Grounding Document constraints.
- The already-selected classes (to detect duplicates and cross-schema conflicts).
- The NL summaries and metadata of the top-3 candidates.

The LLM returns: selected candidate index, confidence tier, whether adaptation is needed, adaptation notes, and whether any conflict was detected. If no candidate is suitable, the slot is downgraded to GAP.

Stage 4 — Adaptation, Generation & Assembly

FR-4.1 Class Adaptation (MEDIUM Confidence)

For MEDIUM confidence slots, the system loads the full raw class JSON from PostgreSQL and uses the LLM to adapt it to the new problem. Strict adaptation constraints:

- **Permitted changes:** Field values in Condition fields (C2), filter values in Criteria (C4), arithmetic sub-expressions in ComponentExpr (C3) where the formula structure is preserved, GETGROUPFROMSCHEMA2 source schema references where a semantically equivalent schema is available.
- **Prohibited changes:** Component Type (MATH / FETCHFROMSCHEMA / SUMFROMSCHEMA), the fundamental structure of ComponentExpr for cross-schema references, SettlementType granularity (must match the Grounding Document), removal or addition of components without explicit decomposition slot support.
- All adapted class JSON must pass 4C structural validation before proceeding to assembly.

FR-4.2 Net-New Class Generation (GAP Slots)

For GAP slots, the LLM generates a new class JSON from scratch, using the closest corpus primitives as structural templates. Requirements:

- The generated class must follow the exact same JSON structure as the template — Root, class node, Conditions, Components, Criteria nesting.
- ComponentExpr syntax must match the conventions observed in corpus classes of the same computation_type.
- Generated classes are automatically assigned LOW confidence and requires_human_review: true regardless of validation outcome.
- Generation is retried up to 3 times on validation failure. Persistent failures escalate to human review with the validation error details.

FR-4.3 4C Structural Validation

Every class (adapted or generated) must pass automated Pydantic validation before assembly. Validation rules:

- **Class level (C1):** SchemaName, DataSource, DateAggregationOn, SettlementType, OutputProcessCode are all present and non-empty.
- **Conditions (C2):** All condition fields have a val. Fields with list values must have equality='IN'. GETGROUPFROMSCHEMA2 expressions must parse correctly to (SchemaName, ColumnName, filter_json).
- **Components (C3):** Each component has ComponentName, ModelID, Type (from allowed enum), ComponentType, ComponentExpr. If Type is FETCHFROMSCHEMA or SUMFROMSCHEMA, ComponentExpr must be valid JSON. If Type is MATH, ComponentExpr must be non-empty string.
- **Criteria (C4):** Each component must have a Criteria block with at least one #Criteria#N row. Each criteria row must have at least one field condition.

FR-4.4 Dependency Resolution & Schema Assembly

- Build a local DAG from the full set of adapted/generated classes using the cross-schema references extracted during parsing.
- Run topological sort (NetworkX) to determine execution order. Circular dependency detection must raise a pipeline error before assembly proceeds.
- Assemble the final schema JSON in the root structure: `[ { "title": "Root", "technicalName": "Root", "children": [ SchemaGroupName, SchemaClass ] } ]`.
- Schema class nodes must be numbered sequentially: `#SchemaClass#1`, `#SchemaClass#2`, etc., in topological order.

Stage 5 — Engine Validation & Human Review

FR-5.1 Engine Dry-Run Validation

- Submit the assembled schema JSON to the existing schema processing engine using the same Event_Input structure (TenantID, Process, SchemaKey, SourceDataMap) used in production.
- Use a synthetic SourceDataMap with known input values and pre-computed expected output (matching the schema family).
- Compare engine output field-by-field against expected output. All discrepant fields are recorded in the confidence report.
- Dry-run execution must be isolated — it must not write to any production data store.

FR-5.2 Confidence Report

The system generates a structured confidence report for every pipeline run:

- Overall confidence level (HIGH / MEDIUM / LOW) — downgraded by any LOW-confidence class or dry-run failure.
- Engine dry-run result: PASS or FAIL with list of discrepant fields.
- Per-class table: slot_id, class name, source (corpus / adapted / generated), confidence_tier, adaptation_notes, requires_human_review flag.
- Any detected conflicts or circular dependency errors.

FR-5.3 Human Review Dashboard

A Streamlit v1 review interface presents the draft schema and confidence report to a domain reviewer:

- Full draft schema JSON with syntax highlighting.
- Color-coded confidence table (green = HIGH, amber = MEDIUM, red = LOW/GAP) with per-class review flag.
- For MEDIUM/LOW classes: side-by-side diff between the source corpus class and the adapted/generated version, with 4C-layer-level annotations.
- Engine dry-run result prominently displayed; discrepant output fields highlighted.
- Three reviewer actions: Approve as-is, Edit and approve (inline JSON editor with live 4C validation), Reject (with required rejection reason).

FR-5.4 Feedback Loop

- Approved schemas are written back to all three stores: PostgreSQL (raw JSON + parsed class records), NL summary regenerated, re-embedded in Pinecone, dependency edges added to Neo4j.
- Reviewer edits are logged with: original generated version, edited version, reviewer ID, edit timestamp. This log is the primary data source for future fine-tuning.
- Rejected schemas are logged with rejection reason for pipeline quality analysis.

7. Non-Functional Requirements

7.1 Correctness & Safety

- Zero 4C structural violations may reach the human review gate. The Pydantic validator is the absolute gate before assembly.
- The pipeline must never proceed past the ambiguity check with an unresolved ambiguity in the Grounding Document. This is a hard halt, not a warning.
- Circular dependencies in the assembled schema must always raise a pipeline error — the engine's behavior on circular schemas is undefined.
- Generated schemas must never be deployed to production without explicit human reviewer approval. The approval gate is mandatory.

7.2 Performance

- Corpus ingestion (30 schemas, initial run): complete within 6 hours.
- Incremental ingestion (1 new approved schema): complete within 15 minutes.
- End-to-end pipeline (7-class schema, all MEDIUM confidence): complete within 10 minutes excluding human review time.
- Vector retrieval latency per slot: < 500ms.
- LLM calls per pipeline run: < 30 total (configurable cap to control API cost).

7.3 Security & Data Governance

- Schema JSON files contain sensitive financial business logic. They must not be transmitted outside approved infrastructure boundaries or logged in plain text in external services.
- All LLM API calls use the Anthropic Claude API exclusively. API keys are managed through a secrets manager — never stored in code or environment files in version control.
- The review dashboard must require authenticated access (internal SSO or equivalent).
- All pipeline runs, inputs, outputs, and reviewer decisions must be retained in the audit log for a minimum of 5 years per financial data retention policy.

7.4 Observability & Maintainability

- Every pipeline stage emits structured JSON logs containing: stage name, input hash, output hash, LLM call count, retrieval scores, validation results, duration.
- Confidence thresholds, top-K values, model identifiers, retry limits, and all configurable parameters are externalized in a configuration file — never hardcoded.
- Each pipeline stage must be independently executable with mock inputs for unit testing.
- A Prefect dashboard shows pipeline run history, stage durations, failure rates, and escalation counts.

8. Delivery Plan

Phase	Timeline	Deliverables	Gate / Dependency
Ph 1	Wks 1–2	Stage 0 complete: all 30 schemas annotated, parsed into PostgreSQL, dependency graph in Neo4j, NL summaries generated, vectors in Pinecone.	SME availability for annotation sessions (2–3 days required)
Ph 2	Week 3	Stage 1: Problem intake and Grounding Document generation. Tested against 5 known problems where expected decomposition is known.	Phase 1 corpus fully indexed
Ph 3	Week 4	Stage 2: Problem decomposition. Stage 3: Retrieval and selection. Retrieval precision@5 measured on 30-schema test set. Threshold tuning.	Phase 2 Grounding Doc validated
Ph 4	Wks 5–6	Stage 4: Adaptation, gap generation, 4C validation, dependency resolution, schema assembly. End-to-end test on PO Validation schema family only.	Retrieval precision@5 > 0.80
Ph 5	Week 7	Stage 5: Engine dry-run integration. Confidence report. Streamlit review dashboard v1. UAT with one domain reviewer.	Engine dry-run API accessible in test environment
Ph 6	Wk 8+	Iteration on UAT feedback. Extension to additional schema families. Monitoring and alerting. Corpus growth tracking.	UAT sign-off from domain reviewer

9. Risks & Mitigations

Risk	Severity	Mitigation
SME unavailability stalls corpus annotation (Stage 0)	**High**	Pre-book SME sessions before project kickoff. Provide annotation templates to minimize per-session effort. Annotation can proceed schema-by-schema; partial corpus still enables Phase 2–3 development.
GETGROUPFROMSCHEMA2 expressions too complex for automated parsing	**Medium**	Build a dedicated parser for this expression type. Test against all 30 schemas in Stage 0. Flag unparseable expressions for manual annotation rather than failing the entire ingestion.
LLM-generated class JSON fails 4C validation repeatedly (GAP slots)	**High**	Use at least 3 corpus examples as structural templates in the generation prompt. Feed Pydantic error details back into each retry prompt. After 3 failures, escalate to human with full error context rather than passing invalid JSON.
Retrieval precision below target on initial measurement	**Medium**	Iteratively tune: (1) embedding text composition (add/remove metadata fields), (2) similarity threshold, (3) metadata filters. voyage-finance-2 domain tuning expected to significantly outperform general embeddings.
Engine dry-run reveals semantic errors not caught by 4C validation	**High**	Build synthetic test cases with known expected outputs for each schema family before Phase 4. Expand test suite with every approved schema. Semantic errors surface patterns for Pydantic rule additions.
Corpus grows inconsistently — approved schemas not systematically added	**Medium**	Feedback loop is automated (Stage 5.4). Human action required only to approve; ingestion is automatic post-approval. Corpus growth metric tracked in Prefect dashboard.

10. Open Questions & Decisions Required

- **Confidence thresholds:** Should HIGH/MEDIUM/LOW thresholds be global or per computation type? A workflow_assembly class may need a stricter threshold than a fetch_lookup class. Decision needed before Phase 3.
- **Decomposition confirmation gate:** Should the system always show the decomposition (slot list) to the user for confirmation before retrieval begins, or only for high/medium complexity problems? Always showing adds a human step but catches decomposition errors early.
- **Corpus versioning:** When a previously approved schema is corrected (business rule change), how is the old version handled in the vector store and graph? Should old versions be archived or fully replaced?
- **LLM cost cap:** The proposed < 30 LLM calls per pipeline run cap needs validation against actual token usage for a 19-class schema. If complex schemas require more, a tiered cost policy is needed.
- **Dashboard access control:** In v1, is the review dashboard accessible to all domain engineers or restricted to senior SMEs? A tiered access model (junior can approve HIGH confidence only; senior can approve all) is possible but adds v1 complexity.
- **Test case ownership:** Who is responsible for creating and maintaining the synthetic test cases used in engine dry-run validation? These require domain expertise to construct correct expected outputs.

Appendix — Computation Type Taxonomy

The controlled vocabulary of computation types used throughout the system. All decompositions, retrievals, and adaptations are constrained to this enum.

Computation Type	Primary Component Types	Description & SC00004 Example
sequential_math_chain	**MATH only**	A class that performs an ordered chain of arithmetic operations where each output feeds as input to the next. Example: PO_ItemCalculation — Qty × Rate → Discount → AddlCharge → Basevalue → TaxValue → DocumentValue → ExchangeRate → AmountInLocalCurrency.
fetch_lookup	**MATH only (pass-through)**	A class that reads master data values from its DataSource with no arithmetic. Outputs pass raw field values to downstream classes. Example: CostAllocationMaster — reads CNT and PERC allocation values from CA.
cross_schema_fetch	**FETCHFROMSCHEMA**	A class that pulls a single computed value from an upstream schema class using a join key specification. Example: POCostAllocation1 — fetches CostAllocationParameterValue from CostAllocationMaster.
cross_schema_sum	**SUMFROMSCHEMA**	A class that aggregates (sums) values from an upstream schema class across a join key. Example: TCostAllocationValue — sums CostAllocationParameterValue from POCostAllocation1 by CostAllocationMethod.
multi_source_allocation	**MATH + FETCHFROMSCHEMA + SUMFROMSCHEMA mixed**	A class that combines values from multiple upstream sources with arithmetic to produce a final allocated output. Example: POCostAllocation — fetches TCostAllocationValue, sums from POCostAllocation1, sums from PO_ItemCalculation, then computes CostAllocationParameterPerc and final AmountInLocalCurrency.
group_expansion_lookup	**MATH only (pass-through) + GETGROUPFROMSCHEMA2 in Conditions**	A class that uses GETGROUPFROMSCHEMA2 in its Condition fields to dynamically expand a dimension (LOB, Function, etc.) from another schema, creating one

		output row per expanded group member.
workflow_assembly	**MATH (constant) + GETGROUPFROMSCHEMA2 in Conditions**	A class that assembles workflow routing records by expanding activity codes, names, dependencies, and statuses from an approval strategy schema (AP) and LOBs from a cost allocation schema. Example: POWF.